

SHAH ABDUL LATIF UNIVERSITY
Department of Computer Science

Assignment 4 – API Testing

Functional and Performance Testing of a REST API using Postman and Apache JMeter

Student Name:	Muqadas Shahid
Course / Program:	Computer Science
University:	Shah Abdul Latif University
Testing Tools Used:	Postman, Apache JMeter 5.6.3
API Under Test:	JSONPlaceholder REST API
Base URL:	https://jsonplaceholder.typicode.com
Submission Date:	August 16, 2026

Table of Contents

1. Introduction	3
2. Objectives.....	3
3. Tools and Technologies.....	3
4. API Under Test	4
5. API Endpoints Tested	4
6. Postman Test Implementation.....	4
7. Postman Test Cases / Assertions	5
8. Postman Execution Results	7
9. CRUD API Testing	8
10. JMeter Performance Testing.....	10
11. JMeter Test Configuration	10
12. JMeter Performance Results.....	11
13. Performance Analysis.....	12
14. Test Evidence	12
15. Defects / Issues Found	12
16. Overall Test Summary	13
17. Conclusion	13
18. Recommendations	13
19. Appendix	13
20. Executive Summary.....	14

1. Introduction

Application Programming Interface (API) testing is a type of software testing that verifies whether an API meets its functional expectations in terms of correctness, reliability, performance, and security. Unlike UI testing, API testing interacts directly with the application logic and data layer by sending requests to endpoints and validating the structure and content of the responses, along with status codes and response times.

API testing is important because APIs form the communication backbone of modern web and mobile applications. Validating APIs early helps identify defects before they propagate to the user interface, reduces integration risk, and enables automated, repeatable regression testing.

The purpose of this assignment is to gain hands-on experience with two industry-standard API testing tools: Postman, used for functional testing and automated assertion scripting, and Apache JMeter, used for performance and load testing. The publicly available JSONPlaceholder REST API was used as the system under test.

Functional testing verifies that an API behaves correctly according to its specification — checking status codes, response structure, and returned data for individual requests. Performance testing, in contrast, evaluates how an API behaves under concurrent or repeated requests, measuring response time, throughput, and error rate. Both testing types are complementary and were performed as part of this assignment.

2. Objectives

- Validate the core API endpoints of the JSONPlaceholder REST API.
- Verify that each endpoint returns the correct HTTP status code.
- Validate that API responses are returned in the correct JSON format.
- Verify that response data contains the expected fields and values.
- Automate API assertions using Postman test scripts.
- Measure API response performance using Apache JMeter.
- Identify any errors that occur during load testing.

3. Tools and Technologies

The following tools and technologies were used to complete this assignment:

Tool	Version	Purpose
Postman	Latest (Cloud/Desktop)	Functional API testing and automated assertion scripting
Apache JMeter	5.6.3	Performance and load testing of the API

Tool	Version	Purpose
JSONPlaceholder REST API	Public API	System under test (fake online REST API for testing)

4. API Under Test

JSONPlaceholder is a free, publicly available fake REST API commonly used for testing, prototyping, and learning purposes. It simulates a typical resource-based API by providing collections such as posts, comments, albums, photos, todos, and users, and supports standard HTTP methods (GET, POST, PUT, PATCH, DELETE).

Base URL: <https://jsonplaceholder.typicode.com>

This assignment focuses specifically on the /posts resource, which was used to demonstrate CRUD (Create, Read, Update, Delete) operations.

5. API Endpoints Tested

Method	Endpoint	Purpose	Expected Status
GET	/posts	Retrieve all posts	200
GET	/posts/1	Retrieve a single post	200
POST	/posts	Create a new post	201
PUT	/posts/1	Update an existing post	200
DELETE	/posts/1	Delete a post	200

6. Postman Test Implementation

A dedicated Postman collection named "Assignment 4 - API Testing" was created to organize all requests used in this assignment (Figure 1).

The collection contains five requests, one for each endpoint under test: GET - All Posts, GET - Single Post, POST - Create Post, PUT - Update Post, and DELETE - Delete Post.

Each request uses the appropriate HTTP method (GET, POST, PUT, DELETE) and targets the corresponding JSONPlaceholder endpoint. For each request, an automated Post-response script was written using Postman's `pm.test()` JavaScript API to validate the response programmatically. These scripts perform the following categories of validation:

- JSON validation – confirming that the response body is valid JSON (`pm.response.to.be.json`).
- Status-code validation – confirming that the HTTP status code matches the expected value for the operation (e.g., 200 for GET/PUT/DELETE, 201 for POST).

- Response structure validation – confirming that the response contains the required fields (id, title, body, userId).
- Data validation – confirming that specific field values (such as post ID, title, or body) match what was sent or expected, including verification that PUT updates were applied correctly.
- Response-time validation – confirming that each response completes within an acceptable time threshold (under 15 seconds).

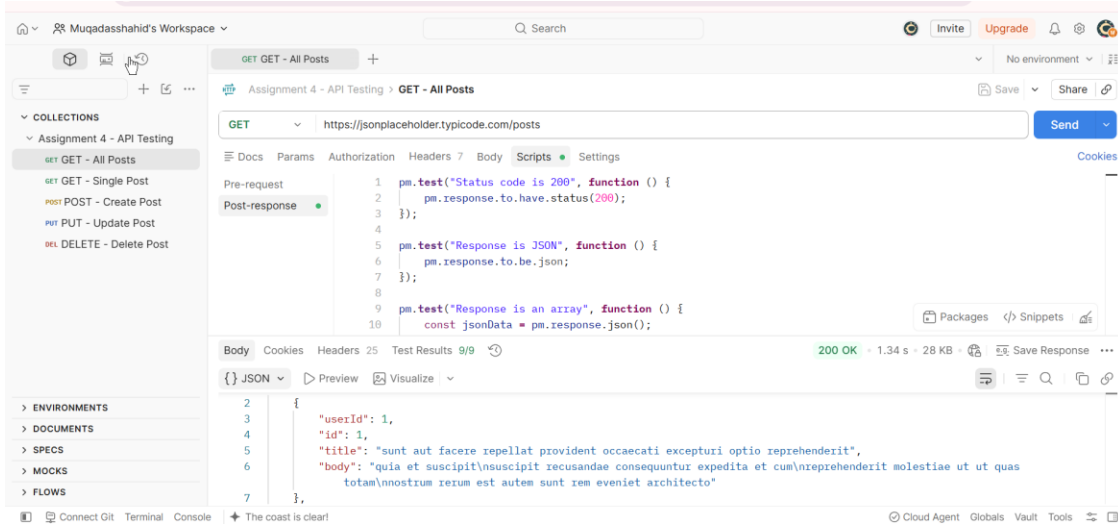


Figure 2: GET – All Posts request showing the Postman post-response test script and the returned JSON response.

7. Postman Test Cases / Assertions

The table below summarizes the automated test assertions implemented for each request in the Postman collection.

GET - All Posts

Test Area	Validation	Expected Result	Status
Status Code	Status code is 200	200	Passed
Format	Response is JSON	Valid JSON	Passed
Structure	Response is an array	Array	Passed
Content	Response contains posts	Non-empty array	Passed
Fields	First post contains id	Field present	Passed
Fields	First post contains title	Field present	Passed
Fields	First post contains body	Field present	Passed
Fields	First post contains userId	Field present	Passed
Performance	Response time < 15 seconds	Below threshold	Passed

GET - Single Post

Test Area	Validation	Expected Result	Status
Status Code	Status code is 200	200	Passed
Format	Response is JSON	Valid JSON	Passed
Fields	Response contains id	Field present	Passed
Fields	Response contains title	Field present	Passed
Fields	Response contains body	Field present	Passed
Fields	Response contains userId	Field present	Passed
Performance	Response time < 15 seconds	Below threshold	Passed

POST - Create Post

Test Area	Validation	Expected Result	Status
Status Code	Status code is 201	201	Passed
Format	Response is JSON	Valid JSON	Passed
Fields	Response contains id	Field present	Passed
Fields	Response contains title	Field present	Passed
Fields	Response contains body	Field present	Passed
Fields	Response contains userId	Field present	Passed
Data	Title is correct	Matches request	Passed
Data	User ID is correct	Matches request	Passed
Performance	Response time < 15 seconds	Below threshold	Passed

PUT - Update Post

Test Area	Validation	Expected Result	Status
Status Code	Status code is 200	200	Passed
Format	Response is JSON	Valid JSON	Passed
Data	Post ID is 1	ID = 1	Passed
Data	Title was updated	Updated title returned	Passed
Data	Body was updated	Updated body returned	Passed
Data	User ID is 1	userId = 1	Passed
Performance	Response time < 15 seconds	Below threshold	Passed

DELETE - Delete Post

Test Area	Validation	Expected Result	Status
Status Code	Status code is 200	200	Passed
Performance	Response time < 15 seconds	Below threshold	Passed
Result	Response is successful	Status in [200, 204]	Passed

8. Postman Execution Results

All requests in the "Assignment 4 - API Testing" collection were executed using the Postman Collection Runner. The consolidated results, aggregated across all five requests (GET - All Posts: 9 tests, GET - Single Post: 7 tests, POST - Create Post: 9 tests, PUT - Update Post: 7 tests, and DELETE - Delete Post: 3 tests), are as follows:

Metric	Result
Total Tests	35
Passed	35
Failed	0
Skipped	0
Errors	0
Run Duration	2 seconds 105 milliseconds
Average Response Time	338 ms

All automated Postman assertions passed successfully, with zero failures and zero errors recorded across the entire collection run, confirming that the tested endpoints behaved as expected under functional testing conditions.

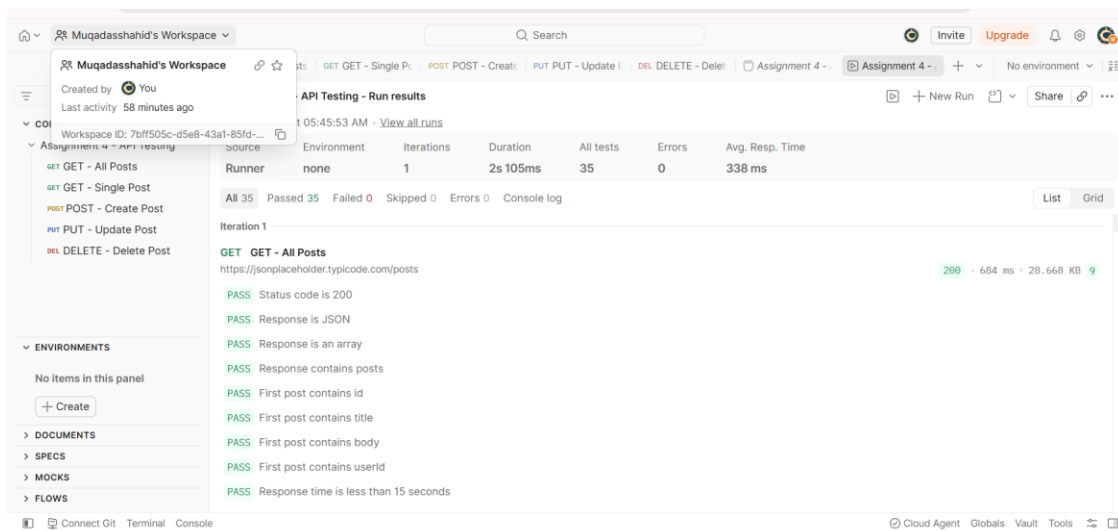


Figure 1: Postman Collection Runner results showing all automated tests passing across the collection.

9. CRUD API Testing

The assignment covered all four fundamental CRUD operations, each mapped to a corresponding HTTP method and validated with dedicated Postman requests and assertions:

- GET (Read) – Used to retrieve data from the API. Two GET requests were tested: retrieving the full list of posts, and retrieving a single post by ID. Both returned HTTP 200 with correctly structured JSON responses.
- POST (Create) – Used to create a new resource. A POST request was sent to /posts with a title, body, and userId in the request payload. The API returned HTTP 201 (Created) along with the newly created resource, including a generated id.
- PUT (Update) – Used to update an existing resource. A PUT request was sent to /posts/1 with an updated title and body. The API returned HTTP 200 with the updated field values reflected in the response.
- DELETE (Delete) – Used to remove a resource. A DELETE request was sent to /posts/1, and the API returned HTTP 200, confirming successful deletion.

POST - Create Post (Status 201)

The screenshot shows the Postman interface for a POST request. The top bar indicates the request is a POST to `https://jsonplaceholder.typicode.com`. The left sidebar shows the 'Test Results' tab selected, displaying 9/9 tests passed. The main area shows the 'Test Results' tab with a list of 9 tests, all of which are 'PASSED'.

Test Results:

- PASSED Status code is 201
- PASSED Response is JSON
- PASSED Response contains id
- PASSED Response contains title
- PASSED Response contains body
- PASSED Response contains userId
- PASSED Title is correct
- PASSED User ID is correct
- PASSED Response time is less than 15 seconds

PUT - Update Post (Status 200)

The screenshot shows the Postman interface for a PUT request. The top bar indicates the request is a PUT to `https://jsonplaceholder.typicode.com/posts/1`. The left sidebar shows the 'Test Results' tab selected, displaying 7/7 tests passed. The main area shows the 'Test Results' tab with a list of 7 tests, all of which are 'PASSED'. A yellow circle highlights the 'Test Results' tab in the sidebar.

Test Results:

- PASSED Status code is 200
- PASSED Response is JSON
- PASSED Post ID is 1
- PASSED Title was updated
- PASSED Body was updated
- PASSED User ID is 1
- PASSED Response time is less than 15 seconds

DELETE - Delete Post (Status 200)

The screenshot shows the Postman interface for a DELETE request. The top bar indicates the request is a DELETE to `https://jsonplaceholder.typicode.com/posts/1`. The left sidebar shows the 'Test Results' tab selected, displaying 1/1 tests passed. The main area shows the 'Test Results' tab with a list of 1 test, which is 'PASSED'.

Test Results:

- PASSED Status code is 200

Figure 3: CRUD API request validation and automated test results for POST, PUT, and DELETE operations.

10. JMeter Performance Testing

Following functional validation in Postman, a basic performance test was conducted using Apache JMeter 5.6.3 to evaluate the response behavior of the GET - All Posts endpoint under repeated requests.

A Test Plan named "API Load Testing - JSONPlaceholder" was created, containing a Thread Group configured to send repeated GET requests to the /posts endpoint. Two listeners were attached to the test plan to capture results:

- View Results Tree – displays each individual sample request and response, allowing visual confirmation that requests completed successfully (Figure 4).
- Summary Report – aggregates statistics across all samples, including average, minimum, and maximum response time, throughput, and error percentage (Figure 5).

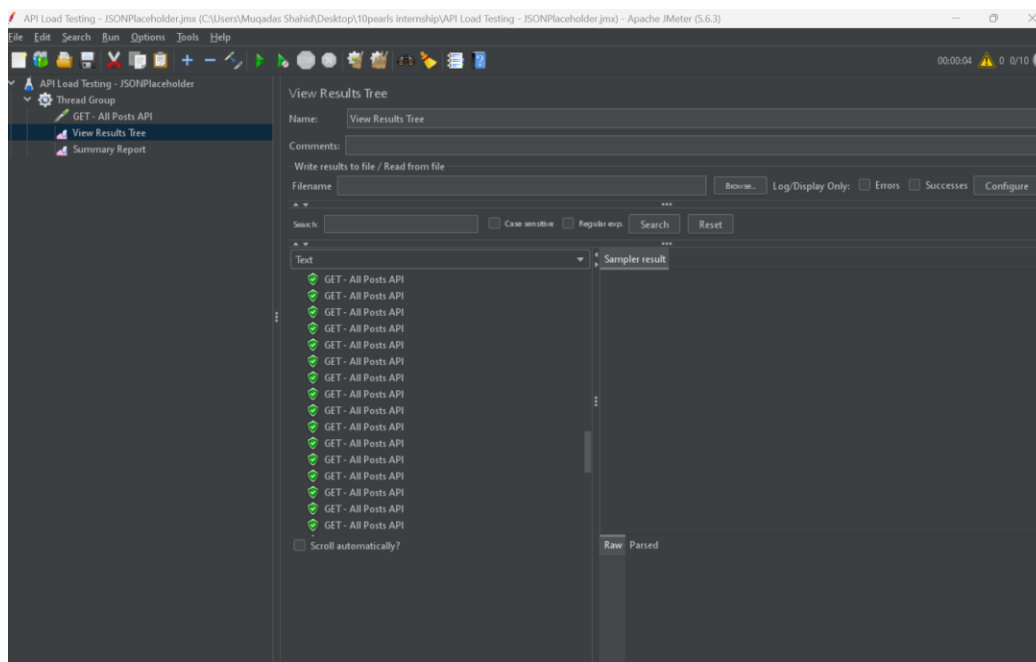


Figure 4: JMeter View Results Tree showing successful (green-checked) GET - All Posts API requests.

11. JMeter Test Configuration

The configuration visible in the JMeter test plan consisted of the following elements:

- Test Plan name: API Load Testing - JSONPlaceholder
- Thread Group containing a single HTTP sampler: GET - All Posts API
- Target endpoint: GET <https://jsonplaceholder.typicode.com/posts>
- Listeners: View Results Tree and Summary Report
- Number of samples executed: 50 (as reported in the Summary Report)

Detailed thread group settings such as the number of threads (users), ramp-up period, and loop count were not visible in the provided screenshots and are therefore recorded as Not Available. The Summary Report confirms that 50 total samples were executed against the GET - All Posts endpoint.

12. JMeter Performance Results

The Summary Report listener produced the following aggregated performance metrics for the 50 samples executed against the GET - All Posts endpoint:

Metric	Result
Samples	50
Average Response Time	118 ms
Minimum Response Time	21 ms
Maximum Response Time	1832 ms
Standard Deviation	280.98 ms
Error %	0.00%
Throughput	10.6 requests/sec
Received KB/sec	296.54
Sent KB/sec	1.42
Average Bytes	approximately 28750 bytes

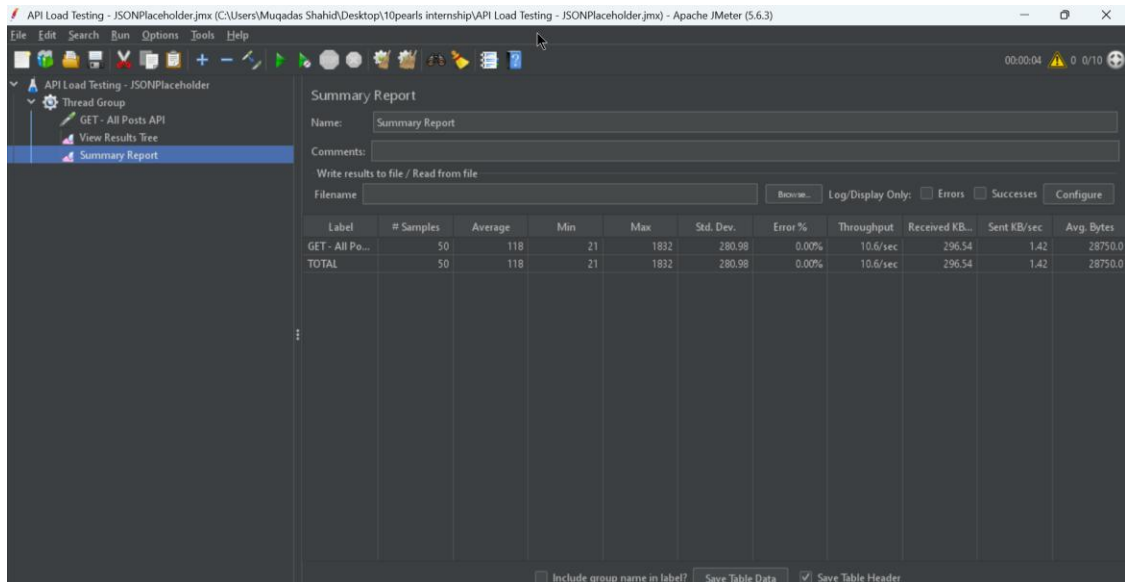


Figure 5: JMeter Summary Report showing response-time, throughput, and error metrics.

13. Performance Analysis

The average response time of 118 ms indicates that, on average, the GET - All Posts endpoint responded quickly and consistently across the 50 samples executed. This is a reasonable response time for a lightweight REST API returning a moderately sized JSON payload.

The minimum response time recorded was 21 ms, representing the fastest observed request, while the maximum response time was 1832 ms. This maximum value is considerably higher than the average, indicating some variation in response time during the test — likely attributable to network latency, connection setup overhead on the first request, or transient conditions on the public JSONPlaceholder service, which is a shared, rate-limited public resource rather than a dedicated performance-testing target.

The standard deviation of 280.98 ms further reflects this variability, showing that response times were not perfectly uniform across all samples, even though the majority of requests completed close to the average value.

The throughput of 10.6 requests per second describes the rate at which the JMeter test plan was able to send and receive requests during the test. Combined with the received data rate of 296.54 KB/sec and sent data rate of 1.42 KB/sec, this indicates a lightweight request/response cycle typical of a JSON-based REST API.

Most importantly, the error percentage of 0.00% indicates that all 50 sampled requests completed successfully, with no failed or errored responses recorded during this test. This confirms that the endpoint remained stable and available throughout the test execution.

These results should be interpreted as a basic performance/load test performed for academic and API-testing learning purposes, and not as a high-volume production load test. No conclusions about production-scale capacity, concurrency handling, or infrastructure scalability can be drawn from this limited sample size.

14. Test Evidence

Screenshots of the Postman collection, automated test scripts, and JMeter results were captured as evidence and are presented alongside the relevant sections throughout this report (Figures 1–5). Additional supporting screenshots are provided in the Appendix.

15. Defects / Issues Found

Functional API Testing

No functional failures were observed. All 35 automated Postman assertions across the five tested requests (GET - All Posts, GET - Single Post, POST - Create Post, PUT - Update Post, and DELETE - Delete Post) passed successfully.

Performance Testing

No request errors were observed. The JMeter Summary Report recorded a 0.00% error rate across all 50 samples of the GET - All Posts endpoint.

16. Overall Test Summary

Postman functional testing: 35 of 35 automated tests passed across five CRUD requests, with 0 failures, 0 skipped tests, and 0 errors, executed in a total collection run duration of 2 seconds 105 milliseconds at an average response time of 338 ms.

JMeter performance testing: 50 samples of the GET - All Posts endpoint were executed with a 0.00% error rate, an average response time of 118 ms, and a throughput of 10.6 requests per second.

Overall outcome: The JSONPlaceholder /posts endpoints demonstrated correct functional behavior across all tested CRUD operations and stable performance characteristics under the basic load conditions applied in this assignment.

17. Conclusion

The tested JSONPlaceholder endpoints successfully passed all implemented functional validations in Postman, and the JMeter load test produced a 0% error rate across the 50 samples executed against the GET - All Posts endpoint. Together, these results demonstrate that the API is functionally correct for the operations tested and performed reliably under the applied load.

It is important to note that this testing was performed in an academic/testing environment using a public, shared demonstration API, and should not be treated as a full production-scale performance benchmark. The results are indicative of the tool usage and testing methodology rather than a certification of production readiness.

18. Recommendations

- Add negative test cases to validate API behavior under invalid conditions.
- Test invalid or non-existent resource IDs (e.g., GET /posts/99999).
- Test invalid payloads (e.g., incorrect data types or malformed JSON).
- Test requests with missing required fields to confirm proper validation/error handling.
- Add authentication and authorization testing for APIs that require secured access.
- Perform larger-scale load and stress tests when evaluating production systems.
- Integrate automated API testing into a CI/CD pipeline for continuous regression testing.

19. Appendix

A. Endpoint List

- GET <https://jsonplaceholder.typicode.com/posts>

- GET <https://jsonplaceholder.typicode.com/posts/1>
- POST <https://jsonplaceholder.typicode.com/posts>
- PUT <https://jsonplaceholder.typicode.com/posts/1>
- DELETE <https://jsonplaceholder.typicode.com/posts/1>

B. Key Postman Test Assertions

- `pm.response.to.have.status(200) / 201` — status-code validation
- `pm.response.to.be.json` — response-format validation
- `pm.expect(jsonData).to.be.an("array")` — structure validation
- `pm.expect(jsonData[0]).to.have.property("id"/"title"/"body"/"userId")` — field presence validation
- `pm.expect(pm.response.responseTime).to.be.below(15000)` — response-time validation
- `pm.expect(pm.response.code).to.be.oneOf([200, 204])` — DELETE success validation

C. Additional Test Result Screenshots

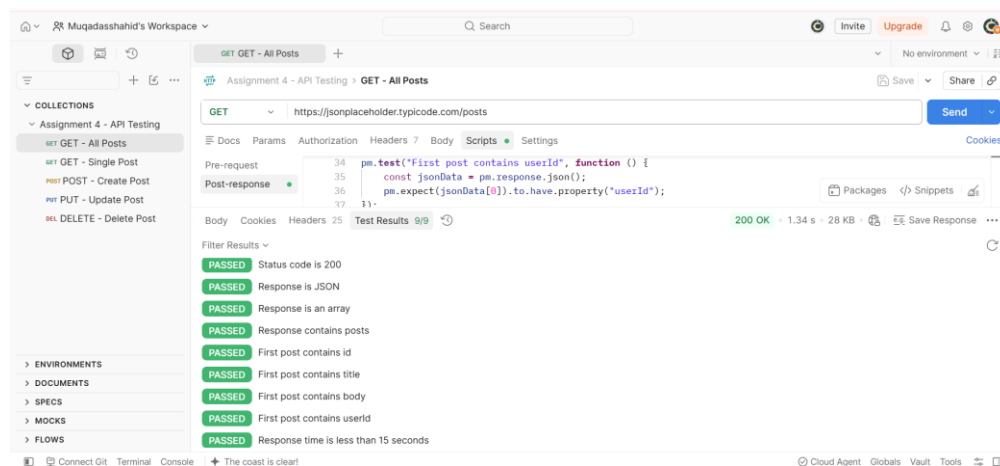


Figure 6: GET – All Posts automated test results (9 of 9 tests passed).

20. Executive Summary

This report documents the functional and performance testing of the JSONPlaceholder REST API (<https://jsonplaceholder.typicode.com>), performed using Postman for functional API testing and Apache JMeter 5.6.3 for performance/load testing.

Postman (Functional Testing)

- 35 of 35 automated tests passed
- 0 failures
- 0 errors
- 5 endpoints tested: GET /posts, GET /posts/1, POST /posts, PUT /posts/1, DELETE /posts/1

JMeter (Performance Testing)

- 50 samples executed against GET /posts
- 0.00% error rate
- 118 ms average response time
- 10.6 requests/sec throughput

Overall, the tested API endpoints met all functional expectations and exhibited stable performance under the basic load conditions applied. This testing was conducted for academic purposes and should not be interpreted as a production-scale benchmark.